

Introduction

Link to the video of the runbook simulation.

postgres-checkup is an open-source diagnostics tool for a deep analysis of a Postgres database health. It detects current and potential issues with database performance, scalability, and security. It also produces recommendations on how to resolve or prevent them. It is considered as an addition to existing monitoring data, augmenting it. A monitoring system will only show current, urgent problems. And postgres-checkup will show sneaking up, deeper problems that may hit you in the future. It helps to solve many known database administration problems and common pitfalls. It aims to detect issues at a very early stage and to suggest the best ways to prevent them.

Project home: <https://gitlab.com/postgres-ai/postgres-checkup>. It contains all the details of how it works and how to use it. As of June 2020, 28 reports are implemented, and 25 of them are available when PG connection (not SSH connection) is used to generate reports, and two snapshots of `pg_stat_statements` (standard Postgres extension for query macro-analysis) data are collected. The full list of reports can be found here: <https://gitlab.com/postgres-ai/postgres-checkup#the-full-list-of-reports>.

GitLab.com database reports can be found here: https://gitlab.com/gitlab-com/gl-infra/production-engineering/-/issues?label_name%5B%5D=postgres-checkup.

When and how reports are generated

Reports are generated by running a short-lived Docker container in <https://ops.gitlab.net/gitlab-com/gl-infra/postgres-checkup/>, connecting to PostgreSQL directly and then posting the results to Snippets and Issues in Infrastructure repository. As of June 2020, postgres-checkup reports for the GitLab.com database are generated twice per week: on Sunday, and on Monday, during peak time (mid-day in Europe), when activity is high. See https://ops.gitlab.net/gitlab-com/gl-infra/postgres-checkup/pipeline_schedules.

postgres-checkup is supposed to analyze all nodes sequentially. The CI variable `HOSTS` is used to define which nodes are to be analyzed – this variable contains a “space-separated” list of IPs. If some node is missing, consider adding it to this list in CI pipeline schedule configuration.

To generate a new report at any time, launch a CI pipeline copying the variables from the CI schedule configuration. Note that two `pg_stat_statements` snapshots are to be collected, with some pause between them. This pause is controlled by the `SNAPSHOT_DISTANCE_SECONDS` CI variable. For the best results, it is recommended to generate reports during peak activity and with `SNAPSHOT_DISTANCE_SECONDS` not less than 600 (10 minutes).

How reports are structured

Technically, each report is generated in multiple forms:

- JSON report files (*.json) — may be consumed by any program or service, or stores in some database.
- Markdown report file (*.md) — the main format for humans, usually contains lists, tables, pictures. Markdown reports are derived from JSON reports.

Additionally, HTML and PDF versions can be generated (when options `--html` and `--pdf` are used). Such report files are derived from markdown format.

Logically, each report is structured as follows:

1. “Observations”: automatically-collected data, with no or small modifications, without treatment. This is to be analyzed by an expert DBA.
2. “Conclusions”: what a DBA may conclude from the Observations, stated in plain English in the form that is convenient for engineers who are not DBA experts.
3. “Recommendations”: action items, what to do to fix the discovered issues.

Both “Conclusions” and “Recommendations” are to be periodically analyzed by engineers who will make decisions on what, how, and when to optimize. Note that as of June 2020, “Conclusions” and “Recommendations” are implemented for specific reports only.

Issue priorities (P1/P2/P3)

The found issues are priorities using three levels of priority:

- P1, “priority one” – critical issues. If not solved in the short term (days, weeks), there are high risks of performance degradation and/or downtime.
- P2 – important issues that negatively affect performance and scalability. Some such issues may transform into P1 ones in the short- or mid-term.
- P3 – potentially bad issues that right now are almost not noticeable, but may become P2 or P1 in mid- or long-term (months, years).

For convenience, at the beginning of aggregated report files (.md, .html, pdf), there is a summary of all P1/P2/P3 issues found.

Most important insights the reports can provide

Configuration and upgrades

Use report **A002 Version Information** to learn which versions of Postgres are currently used. It may be very helpful to control versions and see when (if) minor upgrades were applied on all nodes historically. The report also contains dynamic links to <https://why-upgrade.depesz.com> service, where the bugfixes and improvements between minor versions (those ones that are currently used

and the most up-to-date one) are listed – this can help with the planning of minor upgrades.

A003 Postgres Settings contains the full list of settings. Use it to track what actual values each node had and when. **A006 Postgres Setting Deviations** can help to see if there are differences between nodes – if there are some, it is dangerous because, in the case of failover, we might have unpredicted results. **A007 Altered Settings** serves to show what settings were changed using `ALTER SYSTEM` without using a normal workflow with changes applied in `postgresql.conf` (this is not necessarily the problem, but frequent use of manual `ALTER SYSTEM` may lead to increasing confusion; see also how Patroni controls the configuration of Postgres nodes and how configuration precedence is organized: https://patroni.readthedocs.io/en/latest/dynamic_configuration.html).

A005 Extensions shows extension versions: what version is currently used in the current database (right now the current database is always `gitlabhq_production`), and what version is available. If there is a difference, it is recommended to upgrade the extension to the new version.

Bloat and autovacuum

This topic requires deep Postgres expertise. Reports in group F have Conclusions and Recommendations with links to useful articles, both official PostgreSQL documentation and some selected community articles and blog posts. It is highly recommended to learn how MVCC, and autovacuum are organized in Postgres. The provided articles are good materials to learn.

F001 Autovacuum: Current Settings is a “special look” at autovacuum-related settings (it repeats **A003 Postgres Settings** but is more convenient for analysis of the current autovacuum configuration). Inside the report, there is a list of table-level settings, **Tuned tables**. Important: autovacuum workers that process tables with individual “cost limit” and/or “cost delay” settings have “personal” quotas, while other tables share the “global” quota defined by global parameters. The way how autovacuum configuration is organized in Postgres is quite complicated, and careful reading of documentation is highly recommended (start from here: <https://www.postgresql.org/docs/current/runtime-config-autovacuum.html> and read adjacent articles).

F002 Autovacuum: Transaction ID Wraparound Check is an important report showing the current situation with the “Transaction ID Wraparound” problem. The **Warning** column and Conclusions/Recommendations will show if there is any danger that requires a quick reaction. If it so (“capacity used” levels are » 10%), a manual `VACUUM FREEZE` needs to be executed for specific tables ASAP, and careful analysis of autovacuum activity with proper tuning may also be need.

F004 Autovacuum: Heap Bloat (Estimated) and F005 Autovacuum: Btree Index Bloat (Estimated) indicate how much table and index bloat (estimated!) is in the database. It is helpful to control the bloat and make decisions when to repack indexes and tables using `pg_repack` (see the Runbook on repacking: Reducing table and index bloat using `pg_repack`).

F008 Autovacuum: Resource Usage helps understand if autovacuum can use too many resources (CPU, disk IO).

If you see recommendations to repack some database object, schedule using `pg_repack` to do it. The corresponding runbook:

Health of indexes

Indexes may be bloated, sometimes a lot – often up to 100x (only 1% of the index is “live” data, and 99% is dead, so repacking is recommended ASAP to improve performance and save disk space).

However, this is not the only “bad” thing that may happen to indexes. The other possible issues are:

- Indexes may be *invalid* – usually, it happens in the case of a failed attempt to `CREATE INDEX CONCURRENTLY`. This is not normal and may significantly affect performance. Index rebuilding is recommended ASAP. This is covered in H001 Invalid Indexes.
- Indexes may be *not used*. H002 Unused Indexes shows such indexes, and it analyzes all Postgres nodes altogether, not just the master. Such indexes are to be dropped, to release disk space and slightly improve modifying queries.
- Indexes may repeat other indexes. The H004 Redundant Indexes report covers this topic. With careful analysis, we can consider each index one by one and drop them to save disk space and slightly improve modifying queries.

Additionally, H003 Non-indexed Foreign Keys suggests on the missing indexes that could support foreign keys. Lack of such indexes might not indicate an immediate problem, but it may cause very slow DELETES of the referenced rows.

Query analysis

Query analysis is based on the standard `pg_stat_statements` extension (and, if present, additional quite popular `pg_stat_kcache`, which adds “physical-level” information – details on CPU usage and disk IO).

Each report, K001-K003, have not only metric deltas (such as `calls` or `total_time`) from `pg_stat_statements`, they also provide the following additional “sub-lines” for each big line:

- **metric / observation period in seconds.** Example: **calls per second** (it's QPS, queries per second), or **total_time per second**.
- **metric / calls.** Example: **calls per call** – it's always 1, **total_time per call** – it's latency (average duration of an individual query in the given group).
- **percentage (%)** – how much this line contributes to the whole picture, considering this particular metric.

The basic report is **K003 Top-N Queries by total_time** (N here is configurable, see the CI variables), this is where query groups are analyzed, and Top-N of the most time-consuming groups are provided. To improve scalability, we always need to start from the top of this report. If you see some queries that are »10% by **total_time**, it's a strong signal that optimization is needed. Ping @gl-database for help.

So, the rule of thumb:

Well-optimized database workload never has a high percentage in **total_time** column for individual queries in K003

Why query timing can be bad and what to do:

- The query is slow. It can be seen in **Total time**: if the **total time per call** (in other words, average duration) is high (everything that is more than 100ms should be considered is slow in OLTP context), optimization is needed, ping @gl-database and request optimization. Options will be:
 - if micro-analysis reveals that individual query, being executed without background workload, is also slow, the query itself needs optimization. Use Joe Bot and #database-lab to analyze and verify optimization ideas.
 - if an individual query is fast, but statistics for the corresponding query group look bad, this is a sign of locking issues. Transaction and locking analysis is needed.
- The query is fast (**total time per call** is « 100ms), but the frequency is high (**calls per second** » 1). In this case, reduction of the frequency is recommended. Again, ping @gl-database for that. Options will be:
 - redesign the application code to reduce the frequency (for instance, cache more often; sometimes, high frequency is just a bug),
 - if it's a SELECT happening on the primary, consider offloading it to secondaries.

To see historical data, take **queryid** from the first column (in parentheses) and use it in PromQL when working with Prometheus. For example:

- **calls** data for **queryid=2125684799**: [https://prometheus-db.gprd.gitlab.net/graph?g0.range_input=1d&g0.expr=sum\(rate\(pg_stat_statements_calls%7Bqueryid%3D~%222125684799%22%7D%5B5m%5D\)\)%20by%20\(fqdn\)&g0.tab=0](https://prometheus-db.gprd.gitlab.net/graph?g0.range_input=1d&g0.expr=sum(rate(pg_stat_statements_calls%7Bqueryid%3D~%222125684799%22%7D%5B5m%5D))%20by%20(fqdn)&g0.tab=0)
- **total_time** (in seconds) data for **queryid=2125684799**: <https://promet>

heus-db.gprd.gitlab.net/graph?g0.range_input=1d&g0.expr=sum(rate
(pg_stat_statements_seconds_total%7Bqueryid%3D~%222125684799
%22%7D%5B5m%5D))%20by%20(fqdn)&g0.tab=0

Report **K002 Workload Type ("The First Word" Analysis)** aggregates query groups based on the first word: **SELECT**, **INSERT**, etc. It helps us understand the nature of the workload.

Finally, **K001 Globally Aggregated Query Metrics** shows “the entire picture” for the server. It helps with capacity planning and understanding of the workload globally.

Some key metrics provided in K*** reports, and their derivatives:

- **calls/sec** – QPS, queries per second
- **calls %** – the ratio (by calls) of the given query group. 50% means every second query on the server is a query from this group. In the report **K002 Workload Type ("The First Word" Analysis)**, we can see the percentage of **SELECT**s, in terms of calls.
- **total_time/sec** – how much seconds Postgres on this node spends for query processing every second. 1 sec/sec means “one vCPU on average was busy with query processing of queries from this query group”. Very roughly, if we had just one vCPU, it would be 100% busy with the processing of queries from this group. If we have 96 vCPUs and **total_time/sec** is 48 (for all queries combined, it can be seen in the report **K001**), it may not immediately mean that we have 50% of CPU load – other factors are always playing its role (context switches, background Postgres processes, backups, logging), so we should expect that with 48 / 96 numbers, the load will much higher than 50%. This may help understand the workload and perform capacity planning based on the dynamic analysis provided by K*** reports.
- **total_time/call** – this is the average duration of the query, its “latency” (technically, it should be the same as **mean_time** in raw **pg_stat_statements** data). 100ms should be considered as slow for OLTP environment (for non-user-facing queries, exceeding 100ms is okay, for user-facing ones, it’s definitely not).
- **total_time %** – this is the key indicator of how “heavy” the given query group is. If it’s 50%, then every second second (sorry for the tautology) was spent to process queries from this group. It’s definitely not normal. Any group which has > 10% is to be considered as “heavy” and should be optimized. In the report **K002 Workload Type ("The First Word" Analysis)**, we can see the percentage of **SELECT**s, in terms of timing spent (how heavy the **SELECT**s are) – compare it to **calls %** to have interesting insights of how the workload is organized.
- **rows/call** how many rows retrieved or affected by the statement on average.
- **shared_blks_hit**, **shared_blks_read**, **shared_blks_dirtied**, and **shared_blks_written** provide useful insights such as:

- how effective the buffer pool is for every particular query group (in K003), for SELECTs or other types of queries (K002), or in general (K001),
- if we have issues with generating a lot of writes for SELECTs (may happen even on replicas, indicating that hint bits are outdated - autovacuum cannot catch up promptly),
- if queries read too many buffers to deal with too few rows (signaling that queries suffer from bloat and inefficiency of bloat indexes OR indexes are sub-optimal filtering on-the-fly is often applied),
- and so on.